

Path 2 Cyber

Graduation Capstone Project

Mini SOC & Detection Lab

A Kubernetes-Deployed Security Operations Center Laboratory for
Security Monitoring, Detection Engineering, and Incident Response

Supervisor: SMT GROUPS

Year: 2026

Submission Date: 4 August 2026

Authors: Khaleel, Mohammad, Ahmad, Yazan, Murtaja, Abdallah

Contents

1	Executive Summary	4
2	Lab Architecture and Design Decisions	5
2.1	Architecture Overview	5
2.2	Security Monitoring Pipeline	6
2.3	Infrastructure Monitoring Pipeline	6
2.4	Architectural Design Decisions	7
3	Log Onboarding	8
3.1	Overview	8
3.2	Security Log Collection	8
3.3	Infrastructure Metrics Collection	9
3.4	Log Source Inventory	9
3.5	Data Flow Summary	10
4	Detection Rules, Logic, and Tuning	11
4.1	Rule Inventory Overview	11
4.2	Selected Representative Rules	11
4.3	Rules Not Selected for Detailed Review	15
5	Dashboards	16
5.1	Grafana Dashboards	16
5.1.1	SOC Monitoring Dashboard	16
5.1.2	Kubernetes Cluster Monitoring Dashboard	17
5.1.3	Apache Web Server Monitoring Dashboard	18
5.1.4	Ubuntu Infrastructure Monitoring Dashboard	19
5.2	Wazuh Dashboards	20
5.2.1	Capstone SOC – Attack Chain	20
5.2.2	Capstone SOC – Enterprise Monitoring	21
6	Attack Simulation Results and Coverage	23
6.1	Attack Scenario	23
6.2	Detection Coverage	23
7	Challenges Faced and Solutions	25
8	Conclusion and Future Work	26
A	Complete Wazuh Detection Rule Inventory	27

List of Figures

2.1	Overall architecture of the cloud-native SOC laboratory.	6
5.1	SOC Monitoring Dashboard (Grafana).	17
5.2	Kubernetes Cluster Monitoring Dashboard (Grafana).	18
5.3	Apache Web Server Monitoring Dashboard (Grafana).	18
5.4	Ubuntu Infrastructure Monitoring Dashboard (Grafana).	19
5.5	Capstone SOC – Attack Chain dashboard showing the main attack-chain detections and security events.	21
5.6	Capstone SOC – Attack Chain dashboard showing the main attack-chain detections and security events.	22

List of Tables

3.1	Log Source Inventory	10
4.1	Selected Representative Rules – Summary	12
5.1	Dashboard Summary	16
A.1	Complete Custom Detection Rule Inventory (Rules 100200–100344)	27

Chapter 1

Executive Summary

This project presents the development of a portable Mini Security Operations Center (SOC) laboratory designed to provide a practical cybersecurity learning environment. The primary objective was to build a realistic platform that enables students to experience how modern security operations detect, monitor, and investigate cyber attacks within a controlled environment.

The laboratory combines security monitoring, infrastructure monitoring, and incident management into a single platform that can be deployed on a standard computer using lightweight container technology. By integrating multiple open-source security solutions, the project demonstrates how security events generated from different systems can be collected, analyzed, visualized, and investigated through a centralized workflow.

To evaluate the effectiveness of the environment, several realistic attack scenarios were performed against systems within the isolated laboratory. These activities generated security events that were collected and analyzed by the monitoring platform, allowing the team to observe how malicious behavior can be detected, correlated, and presented to security analysts through interactive dashboards. The project also demonstrates how infrastructure monitoring complements security monitoring by showing the operational impact of attacks alongside security alerts.

One of the main goals of the project was to create a laboratory that is both practical and reproducible. By using a lightweight deployment approach, the complete environment can be recreated with relatively modest hardware resources, making it suitable for students, training environments, and cybersecurity education.

Overall, the project demonstrates the complete lifecycle of a modern Security Operations Center, beginning with event collection, continuing through attack detection and monitoring, and concluding with alert investigation and incident management. The resulting platform provides an effective educational environment for developing practical skills in cybersecurity monitoring, detection engineering, and incident response.

Lab Architecture and Design Decisions

2.1 Architecture Overview

Figure 2.1 illustrates the overall architecture of the implemented cloud-native SOC laboratory. The environment was designed around a layered architecture that separates the monitored infrastructure from the monitoring and incident-response components. This separation improves modularity, simplifies management, and closely resembles the architecture of modern enterprise SOC deployments.

At the lowest layer, the monitored infrastructure consists of multiple systems generating both operational metrics and security-related events: a Windows Active Directory server, a client joined to the domain, a DVWA [2] web server connected to a MariaDB database, and an Ubuntu server used to simulate post-exploitation activities and lateral movement.

The central monitoring platform collects telemetry from these systems through two independent data pipelines. Security logs are forwarded to the Wazuh platform, while infrastructure performance metrics are collected by Prometheus through exporters deployed on the monitored hosts. Separating security telemetry from performance metrics allows each platform to specialize in its respective monitoring domain while maintaining interoperability.

Security events detected by Wazuh are forwarded to TheHive for incident management, where alerts can be transformed into investigation cases. Simultaneously, Prometheus stores infrastructure metrics that are visualized through Grafana dashboards, while Alertmanager distributes notifications based on predefined monitoring rules. Slack is integrated with Alertmanager to demonstrate external notification capabilities.

This architecture enables security monitoring, infrastructure monitoring, and incident response to operate simultaneously while sharing information across the SOC ecosystem.

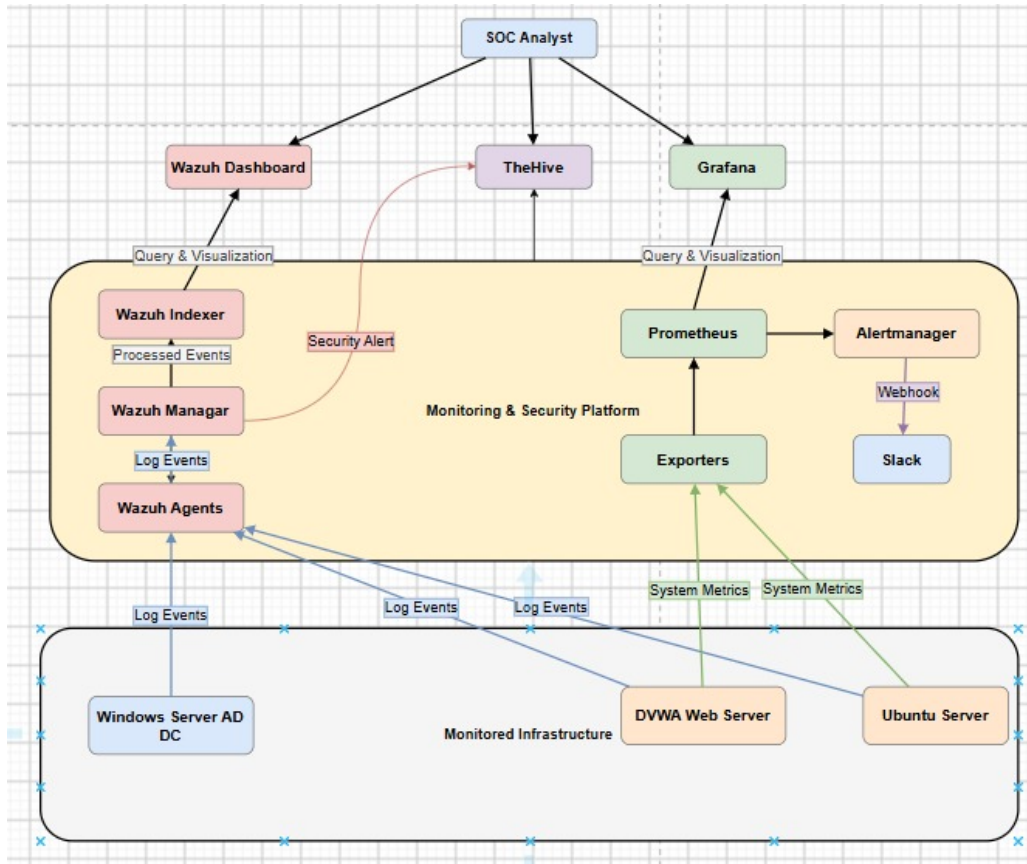


Figure 2.1: Overall architecture of the cloud-native SOC laboratory.

2.2 Security Monitoring Pipeline

The security monitoring workflow begins at the monitored infrastructure, where operating systems and applications continuously generate security-relevant events. These events include authentication logs, Apache access logs, file integrity events, and operating system logs.

Each monitored host runs a Wazuh Agent responsible for collecting these logs and securely forwarding them to the Wazuh Manager. The manager performs log decoding, rule evaluation, and event correlation to determine whether an observed activity represents suspicious or malicious behavior. Detected security events are indexed within the Wazuh Indexer and subsequently visualized through the Wazuh Dashboard. High-severity alerts are additionally forwarded to TheHive, enabling analysts to initiate structured incident response procedures.

2.3 Infrastructure Monitoring Pipeline

Infrastructure monitoring operates independently from the security monitoring pipeline. Node Exporter instances deployed on the monitored Linux systems continuously expose operating system metrics, including CPU utilization, memory consumption, filesystem usage, and network statistics. Prometheus periodically scrapes these metrics and stores them within its time-series database. Grafana retrieves the stored metrics through Prometheus queries to generate real-time dashboards that visualize infrastructure health. Alertmanager continuously evaluates Prometheus alerting rules and sends notifications whenever resource utilization exceeds predefined thresholds; Slack is used as the external notification channel to demonstrate automated infrastructure alerting.

2.4 Architectural Design Decisions

The architecture was intentionally designed using independent but interconnected monitoring components rather than relying on a single monitoring solution.

Wazuh [8] was selected to provide host-based intrusion detection, centralized log analysis, file integrity monitoring, and custom threat detection.

Prometheus [6] and **Grafana** [3] were adopted because they provide efficient collection and visualization of infrastructure metrics, complementing security monitoring by illustrating the operational impact of attacks such as resource exhaustion.

TheHive [7] was incorporated to extend the laboratory beyond simple alert generation by introducing incident case management, representing the investigation phase of a modern SOC.

Kubernetes (Minikube) [1, 4] was chosen as the deployment platform. This choice is kept deliberately brief here, as the container orchestration layer is an enabling deployment detail rather than the focus of this SOC and detection-engineering project: Minikube allows isolated containerized services to be deployed through declarative manifests, and it lets the entire lab – Wazuh, Prometheus, Grafana, and TheHive – be reproduced on a single machine, which directly addressed the hardware constraints described in Chapter 7.

Chapter 3

Log Onboarding

3.1 Overview

To provide comprehensive visibility into both security events and system performance, the laboratory integrates multiple log sources and monitoring components, as illustrated in Figure 2.1. The implemented architecture separates security monitoring from infrastructure monitoring while allowing both pipelines to operate simultaneously. Security events generated by the monitored infrastructure are collected by Wazuh Agents and forwarded to the Wazuh Manager for log analysis, event correlation, and alert generation. Infrastructure metrics are collected through dedicated exporters and periodically scraped by Prometheus for visualization and alerting.

3.2 Security Log Collection

Security monitoring was implemented using Wazuh Agents deployed across the monitored infrastructure. Each agent continuously collects security-related events from its respective host and securely forwards them to the Wazuh Manager for centralized analysis and correlation.

The **DVWA Web Server** provides application-level security logs, including Apache Access Logs, Apache Error Logs, System Logs (Syslog), and File Integrity Monitoring (FIM) events from monitored directories. These logs are used to detect malicious HTTP requests, web server errors, and unauthorized file modifications.

The **Ubuntu Server** contributes System Logs (Syslog) and FIM events, enabling the detection of operating-system activities and unauthorized file creation, modification, or deletion within monitored directories.

The **Windows Domain Controller** Windows Event Logs, including authentication events, account management activities, and other security-related system events. These logs enable centralized monitoring of Windows-based activities within the laboratory environment. As documented in Chapter 4 and Chapter 5, the Windows/AD telemetry pipeline is wired into the ruleset and dashboards.

After receiving the collected events, the Wazuh Manager decodes and analyzes each log using built-in and custom detection rules. The processed events are indexed by the Wazuh Indexer and visualized through the Wazuh Dashboard. High-severity alerts are also forwarded to TheHive to support incident investigation and case management.

3.3 Infrastructure Metrics Collection

Infrastructure monitoring was implemented independently from the security monitoring pipeline, using Prometheus and multiple exporters. **Apache Exporter** was deployed to expose performance metrics from the DVWA web server, including request statistics and Apache server status. **Node Exporter** was deployed on the Ubuntu server to expose host-level metrics such as CPU utilization, memory usage, filesystem statistics, and network activity. In addition, **kube-state-metrics** was deployed to expose the state of Kubernetes resources, including pods, deployments, nodes, and namespaces.

Prometheus periodically scrapes metrics from all configured exporters and stores them as time-series data. Grafana retrieves these metrics from Prometheus to provide real-time dashboards that visualize both application performance and infrastructure health. Alertmanager continuously evaluates predefined alerting rules and generates notifications whenever monitored resources exceed configured thresholds.

3.4 Log Source Inventory

Table 3.1 summarizes every connected log and metrics source documented in the uploaded project materials, together with the collecting agent, destination platform, and monitoring purpose of each source.

Table 3.1: Log Source Inventory

Source	Agent	Collected Logs	Destination	Purpose
DVWA Web Server	Wazuh Agent	Apache Access Logs, Apache Error Logs, System Logs (Syslog), and File Integrity Monitoring (FIM) events	Wazuh Manager → Indexer	Detect malicious HTTP requests, web server errors, and unauthorized file modifications.
Ubuntu Server	Wazuh Agent	System Logs (Syslog) and File Integrity Monitoring (FIM) events	Wazuh Manager → Indexer	Detect operating-system activity and unauthorized file creation, modification, or deletion.
Windows Domain Controller	Wazuh Agent	Windows Event Logs (authentication events, account management activities, and security-related system events)	Wazuh Manager → Indexer	Centralized monitoring of Active Directory authentication and account activity.
DVWA Web Server	Apache Exporter	Apache performance metrics, including request statistics and server status	Prometheus → Grafana	Monitor web application performance and availability.
Ubuntu Server	Node Exporter	Host-level metrics including CPU utilization, memory usage, filesystem statistics, and network activity	Prometheus → Grafana	Monitor infrastructure health and resource consumption.
Kubernetes Cluster	kube-state-metrics	Kubernetes resource state, including pods, deployments, nodes, and namespaces	Prometheus → Grafana	Monitor cluster availability and orchestration health.

3.5 Data Flow Summary

The implemented architecture consists of two independent monitoring pipelines. The **security monitoring pipeline** collects logs from the monitored systems through Wazuh Agents; the collected events are analyzed by the Wazuh Manager, indexed by the Wazuh Indexer, visualized in the Wazuh Dashboard, and forwarded to TheHive whenever incident response is required. The **infrastructure monitoring pipeline** collects metrics using Apache Exporter, Node Exporter, and kube-state-metrics; Prometheus periodically scrapes these metrics, stores them in its time-series database, and provides them to Grafana for visualization, while Alertmanager continuously evaluates alerting rules and distributes notifications when predefined thresholds are exceeded.

Chapter 4

Detection Rules, Logic, and Tuning

4.1 Rule Inventory Overview

The custom Wazuh ruleset for this project defines **25 rules**, with IDs spanning 100200–100344, grouped under the shared **capstone** rule group. The file header notes that the ruleset targets **Wazuh 4.14** and that it must be loaded either as this single combined file, or as six individual per-topic rule files (100200/100220/100240/100260/100300/100340), but **never both at the same time**, to avoid duplicate rule-ID conflicts on manager start-up.

The rules cover six functional areas:

- **Web attacks against DVWA** (100200–100202): SQL injection, automated scanner activity, and generic LFI/RFI/RCE/XSS request patterns.
- **SSH and sudo activity** (100220–100226): failed/invalid/successful SSH logins, direct root login, public-key login, and sudo authentication failure/success.
- **File Integrity Monitoring (FIM)** (100240–100245): modification of `authorized_keys`, `/etc/passwd`, `/etc/shadow`, and `/etc/sudoers`, plus new files under `/opt` and suspicious scripts/binaries under `/tmp`.
- **Windows/Active Directory** (100260–100262): failed logons, new AD user creation, and privileged group membership changes.
- **PowerShell** (100300): suspicious script-block logging (Event 4104).
- **Cross-event correlation rules** (100340–100344): web login brute force, SSH brute force, SSH success following brute force, cross-stage lateral movement (web attack followed by SSH login), and Windows brute force. The complete Wazuh detection rule inventory is provided in Appendix A.

4.2 Selected Representative Rules

From the 25-rule inventory, ten representative rules were selected to illustrate the detection logic across every functional area of the ruleset, spanning both verified and not-yet-verified detections. Table 4.1 summarizes the selection; full detail for each rule follows.

Table 4.1: Selected Representative Rules – Summary

Rule ID	Level	Description	Status
100200	12	SQL injection attempt on DVWA	Verified
100201	12	Automated web scanner activity (UA-based)	Verified
100220	5	Failed SSH login	Verified
100226	10	Successful privilege elevation via sudo	Verified
100240	12	<code>authorized_keys</code> modified (SSH-key persistence)	Verified
100245	10	Suspicious script/binary created in <code>/tmp</code>	Verified
100300	12	Suspicious PowerShell script block (Event 4104)	Verified
100341	12	Correlation: SSH brute force (6+ failed logins/120s)	Verified
100342	15	Correlation: SSH success after repeated failures	Verified
100343	14	Correlation: web attack followed by SSH login (lateral movement)	Verified

Rule 100200 – SQL Injection Attempt (DVWA/Web)

Description DVWA/Web: SQL injection attempt detected from the source IP and requested URL.

Detection Logic Child rule of base rule 31100 (web access log). Matches the requested URL against a PCRE2 pattern covering `UNION SELECT`, `SELECT ... FROM ... information_schema`, `OR 1=1`, `'OR'1'='1`, `SLEEP()`, `BENCHMARK()`, SQL comment sequences (`-`), URL-encoded single quotes (`%27`), and nested `CONCAT(...SELECT...)` patterns.

Data Source Apache access logs from the DVWA web server (Wazuh Agent).

MITRE ATT&CK T1190 – Exploit Public-Facing Application.

Expected False Positives / Tuning The Attack Chain dashboard’s “Web Panel Noise-Reduction Note” documents that the Prometheus Apache exporter repeatedly requests `/server-status?auto`, which otherwise generates noisy, unrelated `rule.id:31516` alerts. The SQL Injection panel therefore applies a visualization-level exclusion filter on `data.url: "/server-status?auto"`; this is a display-only exclusion and does not delete or suppress the underlying events.

Verification Status **Verified** – included in the dashboard’s verified attack-chain rule set (rule.id query on the “Attack Alerts Over Time” and “Alerts by Attack-Chain Stage” panels).

Rule 100201 – Automated Scanner Activity (DVWA/Web)

Description DVWA/Web: automated scanner activity detected from the source IP, based on the HTTP `User-Agent` header.

Detection Logic Child rule of base rule 31100. Matches the `user-agent` field against a PCRE2 pattern for known scanner tool signatures: `sqlmap`, `nikto`, `acunetix`, `dirbuster`, `gobuster`, `wpscan`.

Data Source Apache access logs from the DVWA web server (Wazuh Agent).

MITRE ATT&CK T1595.002 – Active Scanning: Vulnerability Scanning.

Expected False Positives / Tuning The dashboard panel title itself flags this rule as “UN-VERIFIED / Requires Decoder Fix”; the same `/server-status?auto` exclusion filter used for rule 100200 is applied to this panel to reduce Apache-exporter monitoring noise.

Verification Status **Verified** – the dashboard panel explicitly states the rule requires a decoder fix and is not yet verified.

Rule 100220 – Failed SSH Login

Description Capstone/SSH: failed SSH login for a given destination user from a given source IP.

Detection Logic Child rule of base rule 5760 (SSHD authentication failure).

Data Source Syslog / SSHD logs from Linux hosts (Wazuh Agent).

MITRE ATT&CK T1110 – Brute Force.

Verification Status **Verified** – included in the dashboard’s verified attack-chain rule set and has a dedicated “SSH Failed Logins” time-series panel.

Rule 100226 – Successful Privilege Elevation via Sudo

Description Capstone/Sudo: user successfully elevated privileges via sudo.

Detection Logic Child rule of base rule 5402 (sudo command executed).

Data Source Syslog / sudo logs from Linux hosts (Wazuh Agent).

MITRE ATT&CK T1548.003 – Abuse Elevation Control Mechanism: Sudo and Sudo Caching.

Verification Status **Verified** – included in the dashboard’s verified attack-chain rule set and the “Sudo and Privilege Activity” panel; also tagged **thehive** for escalation.

Rule 100240 – authorized_keys Modified (FIM)

Description Capstone/FIM: **authorized_keys** modified on the agent host – possible SSH-key persistence.

Detection Logic Child of the **syscheck** group, base rules 550/554 (file modified/added), with a field match requiring the modified file path to end in **authorized_keys**.

Data Source Wazuh File Integrity Monitoring (syscheck) on monitored Linux hosts.

MITRE ATT&CK T1098.004 – Account Manipulation: SSH Authorized Keys.

Expected False Positives / Tuning The dashboard’s “Persistence / FIM Telemetry Status” note explicitly lists this rule among the three FIM rules treated as verified, in contrast to the **/etc/passwd**, **/etc/shadow**, and **/etc/sudoers** rules, which are explicitly excluded from the verified panels as not yet tested.

Verification Status **Verified**.

Rule 100245 – Suspicious Script/Binary Created in /tmp (FIM)

Description Capstone/FIM: suspicious script or binary created in **/tmp** on the agent host.

Detection Logic Child of the **syscheck** group, base rule 554 (file added), with a field match on file paths under **/tmp/** ending in **.sh**, **.py**, **.pl**, **.php**, **.elf**, or **.bin**.

Data Source Wazuh File Integrity Monitoring (syscheck) on monitored Linux hosts.

MITRE ATT&CK T1059.004 – Command and Scripting Interpreter: Unix Shell.

Expected False Positives / Tuning Confirmed by the “Persistence / FIM Telemetry Status” dashboard note as one of the three verified persistence/FIM detections (alongside 100240 and 100244).

Verification Status **Verified**.

Rule 100300 – Suspicious PowerShell Script Block (Event 4104)

Description Capstone/PowerShell: suspicious script block logged (Event 4104) on the target Windows computer.

Detection Logic Child of the `windows` group, matching Windows Event ID 4104 where the script-block text matches a PCRE2 pattern for `-EncodedCommand`, `FromBase64String`, `DownloadString`, `DownloadFile`, `Invoke-Expression` / `IEX`, `Invoke-Mimikatz`, `-NoP`, or `-WindowStyle Hidden`.

Data Source Windows PowerShell Script Block Logging (Wazuh Agent on Windows 10 client / Domain Controller).

MITRE ATT&CK T1059.001 – Command and Scripting Interpreter: PowerShell; T1027 – Obfuscated Files or Information.

Verification Status **Verified** – the “Windows Field Disclaimer” panel on the Enterprise Monitoring dashboard states that Windows/AD panels, including PowerShell detections, use standard documented field names that “have not been verified against a live indexed alert in this environment.”

Rule 100341 – Correlation: SSH Brute Force

Description Capstone/Correlation: SSH brute force detected from a source IP – 6 or more failed logins within 120 seconds.

Detection Logic Correlation rule using `if_matched_sid` on rule 100220 (failed SSH login), with `frequency="6"` and `timeframe="120"`, grouped by `same_srcip`.

Data Source Derived from rule 100220 (SSHD authentication-failure events).

MITRE ATT&CK T1110.001 – Brute Force: Password Guessing.

Verification Status **Verified** – the dashboard panel title itself reads “SSH Brute-Force Detections (Verified Correlation – Rule 100341).”

Rule 100342 – Correlation: SSH Success After Repeated Failures

Description Capstone/Correlation: successful SSH login following multiple recent failures for the same source IP – likely brute-force success.

Detection Logic Correlation rule requiring rule 100221 (successful SSH login) together with `if_matched_sid` on rule 100220, `frequency="5"`, `timeframe="120"`, grouped by `same_srcip`. This is the highest-severity custom rule in the inventory (level 15).

Data Source Derived from rules 100220 and 100221 (SSHD authentication events).

MITRE ATT&CK T1110 – Brute Force; T1078 – Valid Accounts.

Verification Status **Verified** – the dashboard panel title itself reads “Successful Login After Repeated Failures (Verified – Rule 100342).”

Rule 100343 – Correlation: Possible Lateral Movement

Description Capstone/Correlation: possible lateral movement – a source IP triggered a web-attack alert and then authenticated via SSH to a monitored agent.

Detection Logic Correlation rule requiring rule 100221 (successful SSH login) together with `if_matched_group` on the `web_attack` group, `frequency="2"`, `timeframe="1800"`, grouped by `same_srcip`.

Data Source Derived from web-attack rules (100200–100202) and SSH authentication rule 100221.

MITRE ATT&CK T1021.004 – Remote Services: SSH.

Verification Status **Verified** – the dashboard panel title itself reads “Possible Lateral Movement (100342 Verified / 100343).”

4.3 Rules Not Selected for Detailed Review

The remaining 15 rules in the inventory (100202, 100221–100225, 100241–100244, 100260–100262, 100340, 100344) are listed in Table A.1 with their severity, MITRE mapping, and verification status, but are not expanded into individual rule cards in this chapter, as their detection logic follows the same patterns already illustrated above (base-rule inheritance, field/regex matching, or frequency-based correlation).

Chapter 5

Dashboards

Six monitoring dashboards were developed for the lab: four using Grafana for infrastructure and performance monitoring, and two using the Wazuh Dashboard for security monitoring and threat detection. Table 5.1 summarizes all six; detailed documentation for each follows.

Table 5.1: Dashboard Summary

Dashboard	Platform	Purpose
SOC Monitoring Dashboard	Grafana	High-level overview of the entire lab: cluster/host status, CPU, memory, disk, network, running pods
Kubernetes Cluster Monitoring Dashboard	Grafana	Cluster availability, node/pod/container counts, pod distribution, deployment status, restart counts
Apache Web Server Monitoring Dashboard	Grafana	Apache service health and performance for the DVWA web server
Ubuntu Infrastructure Monitoring Dashboard	Grafana	Operating-system health and resource consumption of the Ubuntu server
Capstone SOC – Attack Chain	Wazuh Dashboard	End-to-end attack narrative: DVWA exploitation → SSH brute force → privilege escalation → persistence → TheHive escalation
Capstone SOC – Enterprise Monitoring	Wazuh Dashboard	Operational, host-level security visibility: Linux authentication, sudo, FIM, Windows/AD, Power-Shell, MITRE coverage, TheHive escalation

5.1 Grafana Dashboards

To provide comprehensive visibility into the laboratory environment, four Grafana dashboards were developed. Each dashboard focuses on a different aspect of the infrastructure, enabling real-time monitoring of system health, Kubernetes resources, and web server performance.

5.1.1 SOC Monitoring Dashboard

This dashboard provides a high-level overview of the entire laboratory environment. It consolidates the most important infrastructure metrics into a single interface, allowing administrators to quickly determine the operational status of the monitored systems.

The dashboard displays:

- Overall status of the Kubernetes cluster, Ubuntu server, and Apache web server.
- CPU, memory, and disk utilization.
- Network transmission and reception statistics.
- Number of running Kubernetes pods.

This dashboard serves as the primary monitoring interface for obtaining an immediate overview of the SOC environment.

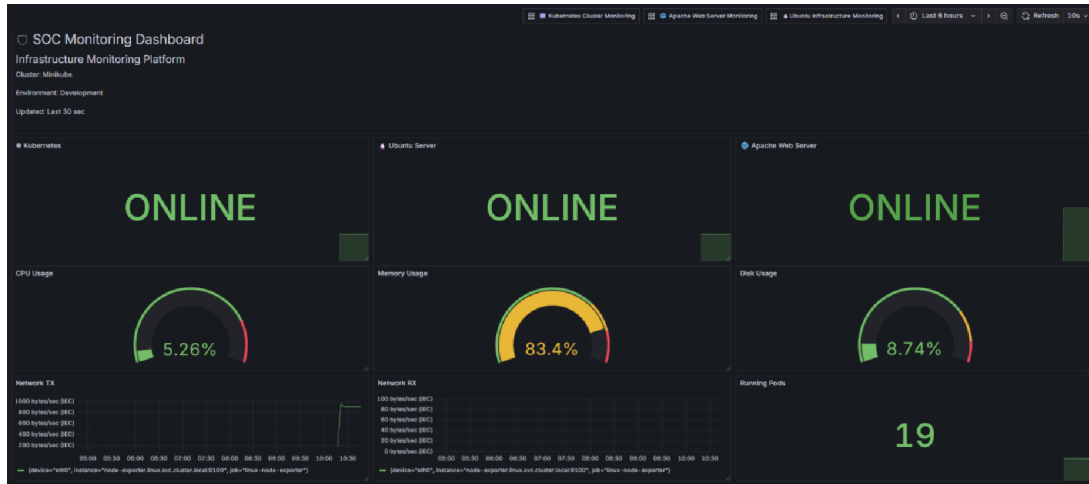


Figure 5.1: SOC Monitoring Dashboard (Grafana).

5.1.2 Kubernetes Cluster Monitoring Dashboard

This dashboard focuses on monitoring the Kubernetes infrastructure and cluster resources.

The dashboard includes:

- Cluster availability status.
- Number of running nodes, pods, and containers.
- Pod distribution across namespaces.
- Deployment ReplicaSet status.
- Pod lifecycle statistics.
- Container restart count.

These metrics help identify infrastructure failures, deployment issues, and resource availability within the Kubernetes cluster.



Figure 5.2: Kubernetes Cluster Monitoring Dashboard (Grafana).

5.1.3 Apache Web Server Monitoring Dashboard

This dashboard provides detailed monitoring of the Apache web server hosting the DVWA application.

The dashboard monitors:

- Apache service availability.
- Server uptime.
- CPU and memory utilization.
- Apache workload and CPU load.
- HTTP request rate.
- Network throughput.
- Response time.
- Busy and idle worker processes.
- Apache scoreboard statistics.

These metrics allow administrators to evaluate the health and performance of the web application and quickly identify abnormal behavior caused by attacks or excessive workloads.

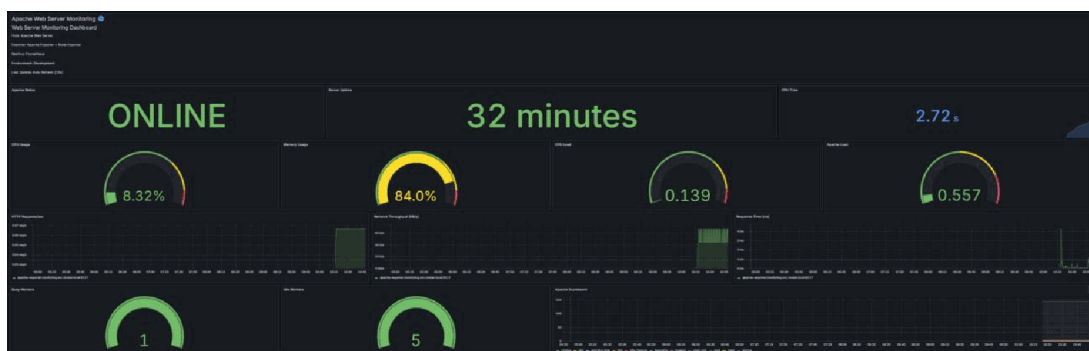


Figure 5.3: Apache Web Server Monitoring Dashboard (Grafana).

5.1.4 Ubuntu Infrastructure Monitoring Dashboard

This dashboard monitors the operating system hosting laboratory services.

The dashboard provides:

- Ubuntu server availability.
- System uptime.
- Number of CPU cores.
- CPU utilization.
- Memory utilization.
- System load.
- Network transmission and reception.
- CPU, memory, and load history over time.

These metrics provide continuous visibility into the server's health and resource consumption, making it easier to detect performance degradation and resource exhaustion.

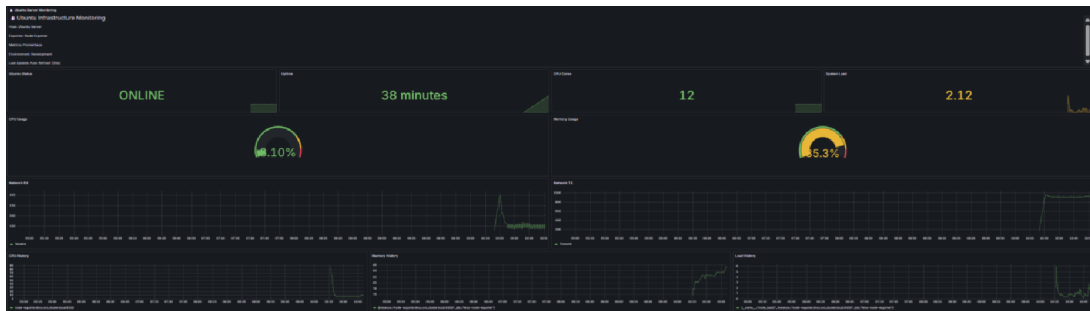


Figure 5.4: Ubuntu Infrastructure Monitoring Dashboard (Grafana).

5.2 Wazuh Dashboards

Two Wazuh dashboards were defined in the uploaded dashboard export (`capstone-all-dashboards-fixed.ndjson`) *Capstone SOC – Attack Chain* and *Capstone SOC – Enterprise Monitoring*. Both are built on a shared `wazuh-alerts-*` index pattern and cross-link to one another. No screenshot image of either Wazuh dashboard was included in the uploaded materials; the draft report’s Section 5.2 (“Wazuh Dashboards”) was left without content or figures.

5.2.1 Capstone SOC – Attack Chain

Purpose Tells the “full attack story from web exploitation to impact and TheHive escalation,” as stated in the dashboard’s own description field.

Data Source `wazuh-alerts-*` index (Wazuh Indexer).

Main Panels

- **Summary metrics:** Total Capstone Alerts; High/Critical Alerts (Capstone); TheHive-Eligible Alerts; Attack-Chain Events; Distinct Source IPs; Distinct Monitored Agents.
- **Attack timeline:** Attack Alerts Over Time (histogram); Alerts by Attack-Chain Stage (donut).
- **Web-attack stage:** SQL Injection Attempts; Automated Scanner Activity (rule 100201, unverified).
- **SSH stage:** SSH Failed Logins; SSH Brute-Force Detections (rule 100341, verified correlation); Successful SSH Logins; Successful Login After Repeated Failures (rule 100342, verified); Possible Lateral Movement (rules 100342/100343, mixed verification).
- **Privilege escalation / persistence stage:** Sudo and Privilege Activity; Persistence/FIM Detections (verified rules 100240, 100244, 100245 only); Suspicious Files by Path.
- **Escalation and context:** TheHive-Eligible High-Confidence Alerts; MITRE Tactics; MITRE Techniques; Top Source IPs; Top Targeted Users; Detailed Chronological Attack-Events Table.
- **Navigation and notes:** link to the Enterprise Monitoring dashboard; Persistence/FIM Telemetry Status note; Web Panel Noise-Reduction Note; shared dashboard filter controls.

Screenshot Figure 5.6 shows the Capstone SOC – Attack Chain dashboard.

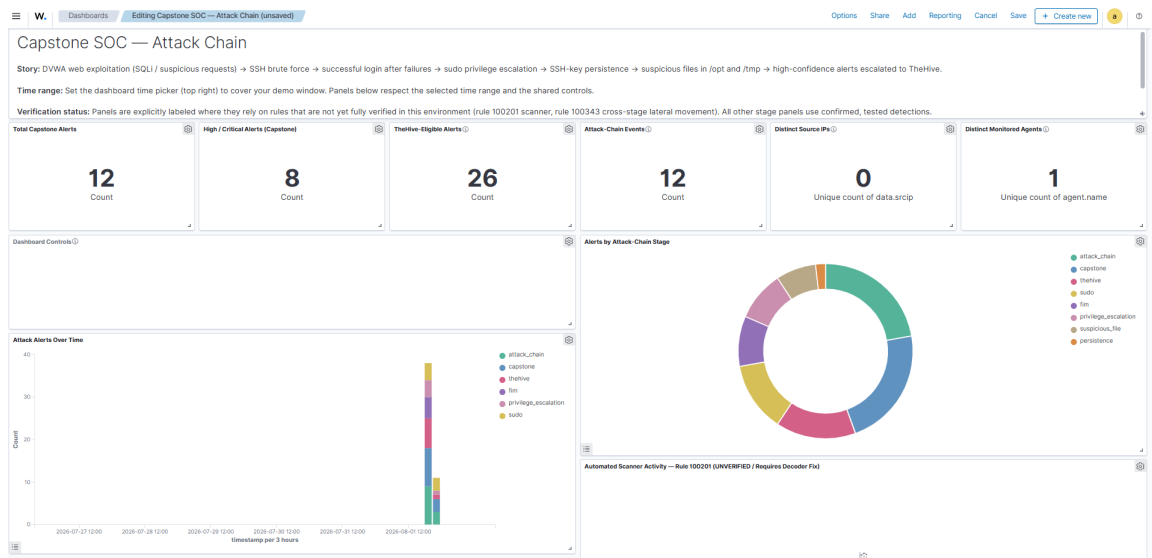


Figure 5.5: Capstone SOC – Attack Chain dashboard showing the main attack-chain detections and security events.

Short Explanation The dashboard is explicitly organized around the attack narrative described in Chapter 6: DVWA web exploitation (SQLi / suspicious requests) leading to SSH brute force, a successful login after failures, sudo privilege escalation, SSH-key persistence, and suspicious files in `/opt` and `/tmp`, with high-confidence alerts escalated to TheHive. Its own on-dashboard markdown notes explicitly flag which panels rely on rules that are “not yet fully verified in this environment” (rule 100201 and rule 100343), stating that “all other stage panels use confirmed, tested detections.”

5.2.2 Capstone SOC – Enterprise Monitoring

Purpose Provides “operational, host-level visibility across Linux authentication, sudo, File Integrity Monitoring, MITRE ATT&CK coverage, and TheHive escalation,” per the dashboard’s own description field.

Data Source `wazuh-alerts-*` index (Wazuh Indexer).

Main Panels

- **Summary metrics:** Total Security Alerts; Capstone Alerts; Critical Alerts (level ≥ 12); High Alerts (level ≥ 10); Active Agents; Authentication Failures; FIM Events; Windows/AD Events (unverified).
- **Severity and host view:** Alerts by Severity (donut); Alerts by Agent/Host.
- **Linux authentication and privilege:** Linux Authentication Activity; Linux Sudo Activity.
- **Windows/AD (unverified telemetry):** Windows Successful vs Failed Logons; Windows Logon Failures Over Time; Top Targeted Windows Users; Account Lockouts; New AD Users; Privileged Group Membership Changes; Password Changes/Resets; Windows Process Creation.
- **PowerShell (unverified telemetry):** PowerShell Activity; Suspicious PowerShell Detections.
- **FIM:** FIM Changes by Host; Top Modified Paths.
- **Coverage and escalation:** MITRE Tactics and Techniques; Alerts Forwarded to TheHive; Recent Critical Alerts (level ≥ 10).

- **Navigation and notes:** link to the Attack Chain dashboard; Windows Field Disclaimer; Severity Band Legend; FIM Rule Telemetry Status note; shared dashboard filter controls.

Screenshot Figure 5.6 shows the Capstone SOC – Attack Chain dashboard.

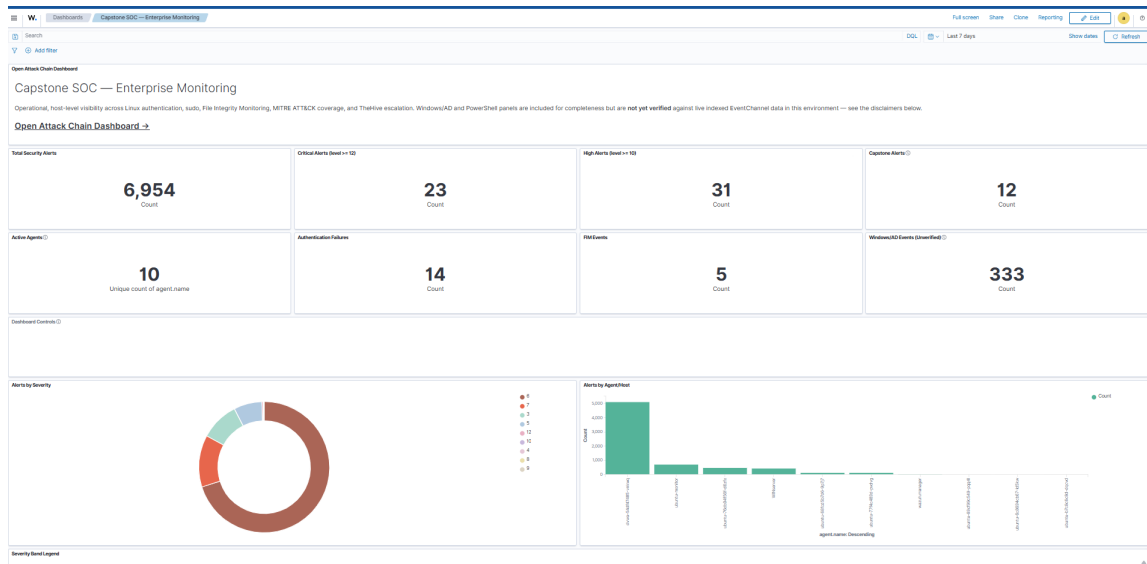


Figure 5.6: Capstone SOC – Attack Chain dashboard showing the main attack-chain detections and security events.

Short Explanation This dashboard broadens the Attack Chain view into general-purpose SOC visibility. Two on-dashboard disclaimers are especially important: the “Windows Field Disclaimer” states that Windows/AD panels “have not been verified against a live indexed alert in this environment,” and the “FIM Rule Telemetry Status” note states that the `/etc/passwd`, `/etc/shadow`, and `/etc/sudoers` FIM rules (100241–100243) are included in the broader `rule.groups:fim` panels but are “not yet tested,” so empty results for those rule IDs should be treated as expected until they are exercised and confirmed.

Chapter 6

Attack Simulation Results and Coverage

The effectiveness of the implemented SOC laboratory was evaluated by executing a realistic multi-stage attack within the isolated laboratory environment. The objective was to demonstrate how security events generated during different stages of an attack can be collected, correlated, detected, and visualized using the implemented monitoring platform.

6.1 Attack Scenario

The simulated attack followed a realistic attack chain consisting of several stages:

1. The attacker targeted the DVWA web application and performed a SQL Injection attack.
2. After gaining access to sensitive information, the attacker used SSH to access the Ubuntu server.
3. The attacker then moved laterally within the laboratory environment by establishing access to another Ubuntu system.
4. To simulate persistence, new files and an SSH key were created inside monitored directories, generating File Integrity Monitoring (FIM) events.
5. Finally, a CPU stress workload was executed to simulate resource exhaustion, allowing the monitoring platform to demonstrate the operational impact of the attack through Grafana dashboards.

6.2 Detection Coverage

Each stage of the attack generated telemetry that was collected and analyzed by the monitoring platform.

- **SQL Injection:** Apache access logs generated by the DVWA web server were collected by the Wazuh Agent and analyzed using custom detection rules, producing security alerts for suspicious web requests.
- **SSH Access:** Authentication events generated during SSH login attempts were collected from the Ubuntu server and correlated with custom SSH detection rules.
- **Lateral Movement:** SSH activity between the laboratory systems demonstrated movement from the compromised host to another monitored Linux system, providing visibility into attacker progression.
- **Persistence:** File Integrity Monitoring detected modifications and newly created files within monitored directories, indicating persistence-related activity.

- **Resource Exhaustion:** The CPU stress workload increased processor utilization, which was immediately visible through the Grafana infrastructure dashboards, demonstrating the operational impact of the simulated attack.

Overall, the attack successfully demonstrated the complete monitoring workflow, beginning with malicious activity against the web application, continuing through system compromise and persistence, and concluding with infrastructure monitoring and security event visualization across the implemented SOC platform.

Chapter 7

Challenges Faced and Solutions

The uploaded draft report documents one major challenge encountered during the development of the laboratory: **limited hardware resources** available to both the development team and prospective users of the lab.

Deploying a separate virtual machine for each security component – Wazuh, Prometheus, Grafana, TheHive, the monitored Windows domain, and the DVWA web server – required significant CPU and memory resources, making the environment difficult to reproduce on standard student laptops.

To resolve this, the laboratory was redesigned around **Docker containers orchestrated by Kubernetes (Minikube)** instead of individual virtual machines. This approach reduced overall resource consumption, simplified deployment through declarative manifests, and enabled students to recreate the complete SOC environment on a single machine with relatively modest hardware requirements. This decision is also reflected in the architectural design choices discussed in Section 2.4.

Chapter 8

Conclusion and Future Work

This project successfully demonstrated the implementation of a containerized Security Operations Center (SOC) laboratory using Kubernetes. The environment integrates security monitoring, infrastructure monitoring, and incident management through Wazuh, Prometheus, Grafana, Alertmanager, and TheHive. The implemented attack scenarios demonstrated the effectiveness of centralized log collection, real-time detection, system monitoring, and incident response within a realistic laboratory environment. Additionally, the containerized architecture reduced hardware requirements, making the platform more accessible and reproducible for students and researchers.

Future enhancements can further improve the laboratory by integrating Cortex with TheHive to enable automated threat analysis and enrichment using external analyzers. Another valuable enhancement is the integration of Threat Intelligence platforms to enrich alerts with indicators of compromise (IOCs) and provide additional context for security investigations. Finally, implementing automated playbooks would enable security incidents to trigger predefined response actions, reduce response time, and improve the overall efficiency of the incident handling process.

Appendix A

Complete Wazuh Detection Rule Inventory

Table A.1 lists every rule in the inventory together with its severity level, a short description, its mapped MITRE ATT&CK [5] technique(s), and its verification status.

The verification status column is derived directly from the project’s own Wazuh dashboard definitions in Chapter 5. Rules explicitly included in the dashboard’s “verified” attack-chain query set, or explicitly labeled *Verified* in a panel title or accompanying note, are marked **Verified**. Rules explicitly labeled *unverified*, *not yet tested*, or *not confirmed against live indexed data* are marked **Not Yet Verified**. Rules that do not appear in any dashboard query or note are marked **Not Confirmed in Evidence**.

Table A.1: Complete Custom Detection Rule Inventory (Rules 100200–100344)

Rule ID	Level	Description	MITRE ATT&CK	Status
100200	12	SQL injection attempt detected (DVWA/Web)	T1190	Verified
100201	12	Automated scanner activity detected (DVWA/Web)	T1595.002	Not Yet Verified
100202	8	Suspicious request pattern – LFI/RFI/RCE/XSS (DVWA/Web)	T1190	Verified
100220	5	Failed SSH login	T1110	Verified
100221	3	Successful SSH login	T1078, T1021.004	Verified
100222	6	SSH login attempt using invalid/non-existent user	T1110	Verified
100223	10	Direct root SSH login succeeded	T1078.001	Not Confirmed in Evidence
100224	3	Public-key SSH login succeeded	T1078	Not Confirmed in Evidence
100225	5	Sudo authentication failure	T1548.003	Verified
100226	10	Successful privilege elevation via sudo	T1548.003	Verified
100240	12	<code>authorized_keys</code> modified – possible SSH-key persistence (FIM)	T1098.004	Verified
100241	12	<code>/etc/passwd</code> modified – possible account tampering (FIM)	T1136.001, T1003.008	Not Yet Verified
100242	13	<code>/etc/shadow</code> modified – critical (FIM)	T1003.008	Not Yet Verified
100243	13	<code>/etc/sudoers</code> configuration modified – critical (FIM)	T1548.003	Not Yet Verified
100244	8	New file created under <code>/opt</code> (FIM)	T1105	Verified
100245	10	Suspicious script/binary created in <code>/tmp</code> (FIM)	T1059.004	Verified

Table A.1 – continued

Rule ID	Level	Description	MITRE ATT&CK	Status
100260	5	Windows failed logon (Event 4625)	T1110	Not Yet Verified
100261	10	New Active Directory user account created (Event 4720)	T1136.002	Not Yet Verified
100262	12	User added to privileged AD security group (Events 4728/4732)	T1098.002	Not Yet Verified
100300	12	Suspicious PowerShell script block logged (Event 4104)	T1059.001, T1027	Not Yet Verified
100340	12	Correlation: web login brute force against DVWA (6+ attempts/120s)	T1110	Not Confirmed in Evidence
100341	12	Correlation: SSH brute force (6+ failed logins/120s)	T1110.001	Verified
100342	15	Correlation: successful SSH login following repeated failures	T1110, T1078	Verified
100343	14	Correlation: possible lateral movement – web attack followed by SSH login	T1021.004	Not Yet Verified
100344	12	Correlation: Windows brute force (6+ failed logons/120s)	T1110	Not Yet Verified

Bibliography

- [1] Cloud Native Computing Foundation. Kubernetes documentation. <https://kubernetes.io/docs>, 2026. Accessed 2026.
- [2] Damn Vulnerable Web Application (DVWA). Dvwa project. <https://dvwa.co.uk>, 2026. Accessed 2026.
- [3] Grafana Labs. Grafana documentation. <https://grafana.com/docs>, 2026. Accessed 2026.
- [4] Kubernetes Authors. Minikube documentation. <https://minikube.sigs.k8s.io>, 2026. Accessed 2026.
- [5] MITRE Corporation. MITRE ATT&CK framework. <https://attack.mitre.org>, 2026. Accessed 2026.
- [6] Prometheus Authors. Prometheus documentation. <https://prometheus.io/docs>, 2026. Accessed 2026.
- [7] StrangeBee. TheHive project documentation. <https://docs.strangebee.com>, 2026. Accessed 2026.
- [8] Wazuh Inc. Wazuh documentation. <https://documentation.wazuh.com>, 2026. Accessed 2026.